Trampolining & Stack Safety

Season: 7 · Advanced FP · Category: Functional · Level: Advanced

Intent

- Convert deep recursion into a loop.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Bouncing instead of stacking trampolines.

When to Use

- Serious FP architecture in Java.
- The design force named in Intent shows up more than once.

When Not to Use

- CRUD apps that don't need effect algebras.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.functional.trampoline;

import java.util.function.Supplier;

/** Stack-safe recursion via trampolining. */
public sealed interface Trampoline<A> permits Trampoline.Done, Trampoline.More {
    record Done<A>(A value) implements Trampoline<A> {}
    record More<A>(Supplier<Trampoline<A>> next) implements Trampoline<A> {}

    static <A> Trampoline<A> done(A a) { return new Done<>(a); }
    static <A> Trampoline<A> more(Supplier<Trampoline<A>> next) { return new More<>(next); }

    default A run() {
        Trampoline<A> cur = this;
        while (true) {
            switch (cur) {
                case Done(var v) -> { return v; }
                case More(var next) -> cur = next.get();
            }
        }
    }

    static Trampoline<Long> safeSum(long n, long acc) {
        if (n == 0) return done(acc);
        return more(() -> safeSum(n - 1, acc + n));
    }
}
```

Modern Java 25 / Functional

- Witness types emulate higher-kinded types.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: functional/trampoline/
```

Pitfalls

- Encodings are educational — production may use libraries.
- Tagless Final needs Kind encoding without HKTs.

Interview Q&A

- Q: When to pick Trampolining & Stack Safety? A: When you need testable effect boundaries or pure state threading.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Convert deep recursion into a loop.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Bouncing instead of stacking trampolines. [0:40] Intent — State the design force in one sentence: Co...